Qwen3-Coder-Next Tier 3
Part 12 Detailed Implementation Specification

Multi-Agent Coordination

Engineering specification for orchestrating specialized agents into a single controlled execution flow

Scope: implementation only. Architecture is assumed finalized. This document defines how coordination should be built, verified, and integrated.

1. Purpose

Part 12 exists to turn a set of individually capable agents into a coordinated software engineering system. Without this layer, the planner, researcher, coder, tester, reviewer, evaluator, and recovery agent all operate as isolated specialists. That produces duplicate work, stale assumptions, contradictory outputs, and awkward retry loops that look busy but do not move the project forward.

The coordination layer solves the control problem between agents. It defines work ownership, task dependencies, parallel execution rules, baton passing, completion checks, conflict resolution, and escalation. It is the component that makes the whole stack behave like a team instead of a pile of prompts sharing the same terminal.

This part is a dependency for every later capability that requires more than one active role. Benchmarking, safety gating, failure recovery, deployment handoff, and large repository work all depend on one reliable rule: the system must know who owns what, what can happen next, and why.

- Primary problem solved: coordination of multiple specialized agents with explicit ownership and handoff semantics.
- Secondary problem solved: prevention of duplicate work, stale context, conflicting edits, and infinite retry loops.
- Dependency value: enables quality gates, recovery flows, and multi-step autonomous execution.

2. Scope

Included in this part:

- Coordination policy engine that decides which agent may act next.
- Task routing, ownership transfer, and dependency tracking.
- Concurrent execution rules for parallel-safe workstreams.
- Shared work item format and agent handoff protocol.
- Conflict handling for competing suggestions, stale state, and out-of-order results.
- Coordination telemetry, trace IDs, and transition logs.

Excluded from this part:

- Model-specific prompting strategies for the individual agents.
- Knowledge graph construction, memory indexing, or retrieval internals.
- Full testing, review, evaluation, or recovery logic beyond the handoff hooks they need.
- Deployment and human approval behavior, except for the coordination handoff into those stages.

3. System Overview

Coordination sits between the top-level orchestrator and the specialized agents. It is the traffic controller that decides which role is allowed to act next, under what evidence, and with what context package. It is not domain intelligence. It is the control plane that keeps domain intelligence from stepping on itself.

```
User Request
  ↓
LangGraph Orchestrator
  ↓
Coordination Layer
  ├─→ Planner Agent
  ├─→ Research Agent
  ├─→ Coding Agent
  ├─→ Testing Agent
  ├─→ Review Agent
  ├─→ Evaluator Agent
  ├─→ Recovery Agent
  └─→ Deployment Agent
  ↓
Aggregated Progress / Next Action / Final Outcome
```

A typical execution sequence is: intake, decomposition, ownership assignment, parallel subtask dispatch, result merging, conflict check, stage transition, and completion confirmation. The coordination layer owns the authoritative map of active tasks, active agents, pending dependencies, and legal transitions.

- Every task enters as a coordination item with explicit status, owner, dependencies, and evidence history.
- Agents never mutate global execution state directly; they submit results back through coordination.
- Parallelism is allowed only when task dependencies do not overlap and work products are isolated.
- All transitions are traceable through a single event log.
- Coordination decides next action, but does not itself perform domain work.

4. Internal Architecture

The internal architecture should stay deliberately small. There should be a policy engine, a registry, a router, a transition validator, an arbitration layer, and an event recorder. Anything more complicated usually means the design has become a hidden business-logic swamp.

```
Coordination Policy Engine
  ↓ validates transitions and concurrency rules
Task Registry / Ownership Table
  ↓ tracks active work items, blockers, outputs
Agent Dispatch Router
  ↓ selects eligible agent(s) and packages context
```

```
Transition Validator
  ↓ checks completion evidence and dependency satisfaction
Merge / Arbitration Module
  ↓ resolves overlapping outputs or conflicts
Event Log / Trace Recorder
  ↓ durable audit trail for debugging and replay
```

Required abstractions:

- WorkItem - a unit of work with an owner, state, dependencies, acceptance targets, and output slots.
- AgentRole - a named capability boundary that defines what actions the agent may take.
- Transition - a state change from one work item stage to another with evidence attached.
- CoordinationDecision - the policy engine output describing the next allowed action.
- CoordinationEvent - durable event record for every assignment, completion, retry, merge, and escalation.

Enforcement rules:

- No agent may advance a work item without meeting the declared entry criteria for the next state.
- No two agents may claim exclusive ownership of the same mutable artifact at the same time.
- Parallel subtasks must declare their merge strategy before dispatch.
- Any failed transition must be recorded with reason, state snapshot, and suggested next action.

5. Project Structure

Recommended directory structure for the coordination subsystem:

```
coordination/
  policy/
  registry/
  routing/
  transitions/
  arbitration/
  events/
  contracts/
  telemetry/
  tests/
```

- policy/ - decision logic that evaluates whether a transition is legal.
- registry/ - task and ownership store for active work items.
- routing/ - chooses which agent or agent group receives a work item.
- transitions/ - implements state changes and entry/exit criteria checks.
- arbitration/ - merges conflicting outputs or escalates unresolved conflicts.
- events/ - append-only coordination event log and replay helpers.
- contracts/ - shared typed schemas for work items, outputs, handoffs, and decisions.
- telemetry/ - counters, spans, and coordination health metrics.
- tests/ - deterministic flow tests, conflict tests, and replay tests.

6. Data Contracts

The coordination layer depends on strict contracts. Loose, humanish blobs are how systems become haunted.

Contract	Required Fields	Purpose	Example Use
WorkItem	id, goal, owner_role, state, dependencies, artifacts, exit_criteria	Represents one unit of coordinated work	Feature request broken into coding and review items
AgentAssignment	work_item_id, agent_role, lease_id, context_ref, ttl	Binds work to one agent at a time	Coding agent receives implementation subtask
TransitionDecision	from_state, to_state, allowed, reason, evidence_ref	Describes legal next move	Test pass enables review stage
MergeResult	inputs, conflict_policy, resolution, output_ref	Combines parallel outputs	Multiple research notes merged into one summary
CoordinationEvent	event_id, type, timestamp, actor, payload, trace_id	Append-only audit record	Assignment, completion, retry, escalation

State examples: work item state should include pending, assigned, blocked, executing, awaiting_merge, awaiting_review, escalated, completed, and aborted. The exact labels matter less than their unambiguous meaning and transition rules.

Message examples should carry trace_id, parent_item_id, artifact_refs, confidence, and evidence_notes so that downstream stages can reason about provenance rather than guessing.

7. State Management

Coordination state belongs to the orchestration runtime, not to individual agents. Agents read snapshots and submit deltas; they do not own the authority to revise global execution state directly.

- Owned state: active work items, ownership leases, transition history, concurrency locks, merge queues, escalation queues.
- Mutable by: coordination policy and transition services only.
- Readable by: all agents through read-only snapshots and scoped task context.
- Persistence: append-only events plus periodically materialized state snapshots for quick restart.
- Consumption: future agents use state snapshots to understand what has already been tried, what is blocked, and which artifacts are authoritative.

A restart must rebuild the current state from events or snapshot plus event tail. That is how coordination avoids losing the plot when the process dies, which it absolutely will, because computers enjoy proving humility.

8. Component Specifications

Each component below has a narrow responsibility. A coordination subsystem that does everything becomes unreadable very quickly.

Component	Purpose	Inputs	Outputs	Failure Conditions	Integration
Policy Engine	Approve or deny transitions	current state, candidate action,	decision, reason, next_allowed_states	missing criteria, stale state, illegal	Feeds transition validator and router

		rules		transition	
Task Registry	Track active work items and leases	assignments, status updates, artifacts	authoritative state snapshot	duplicate ownership, stale lease, lost update	Shared by all agents and orchestration
Router	Select next eligible agent(s)	decision, task metadata, role map	dispatch package	role unavailable, unsafe concurrency, missing context	Plugs into Planner, Research, Coding, Review, etc.
Arbitration Module	Resolve conflicts	parallel outputs, conflict policy	merged result or escalation	incompatible outputs, insufficient evidence	Used after parallel subtasks complete
Event Log	Persist the execution story	all coordination events	replayable history	write failure, ordering violation	Supports restart, audit, and debugging

Pseudocode-level interface expectations:

- evaluate_transition(work_item, candidate_action) -> TransitionDecision
- claim_ownership(work_item_id, agent_role, ttl) -> LeaseResult
- dispatch(work_item_id, agent_role, context_snapshot) -> AssignmentRecord
- record_event(event) -> void
- merge_outputs(work_item_id, outputs, policy) -> MergeResult

9. Implementation Sequence

1. Define the shared coordination contracts first. Without fixed schemas, every other piece will drift.
2. Implement the append-only coordination event model and a replayable state reducer.
3. Build the task registry with ownership leases and dependency tracking.
4. Implement the policy engine so it can approve or reject candidate transitions.
5. Add the router so tasks can be assigned to a single agent or a parallel set of agents when safe.
6. Implement merge and arbitration logic for parallel subtasks and conflicting outputs.
7. Wire coordination telemetry, trace IDs, and execution summaries.
8. Integrate with the LangGraph orchestration loop and the specialized agents.
9. Add replay and restart tests before expanding to edge cases.

Why this order matters: the system should be able to explain and recover from its own coordination decisions before it becomes capable of making more complex ones.

10. Validation Strategy

- Transition tests: verify that legal transitions pass and illegal transitions fail with precise reasons.
- Ownership tests: verify that duplicate leases are rejected and expired leases are reclaimed correctly.
- Concurrency tests: verify that parallel subtasks do not overlap on the same exclusive artifact.
- Merge tests: verify deterministic arbitration for conflicting outputs.
- Replay tests: verify the event log can reconstruct the same state after a restart.
- Integration tests: simulate a feature request flowing across Planner, Research, Coding, Testing, Review, and Evaluation.
- Telemetry checks: ensure every significant transition emits traceable metadata.

Correctness is not “it seems to work.” Correctness is reproducible legal state progression under failure, concurrency, and restart.

11. Deliverables

- Coordination contracts package
- Policy engine module
- Task registry module
- Router module
- Arbitration/merge module
- Append-only event log and replay utility
- Telemetry counters and trace schema
- Integration tests and replay fixtures

Every deliverable should be usable in isolation and also compose cleanly with the rest of the runtime.

12. Acceptance Criteria

- The system rejects any transition that lacks declared entry criteria or required evidence.
- A single work item cannot be owned by more than one exclusive agent at a time.
- Parallel dispatch is permitted only when the dependency graph shows no conflicting exclusive resources.
- After restart, the coordination state reconstructed from the event log matches the pre-restart snapshot.
- Conflicting outputs are either deterministically merged or explicitly escalated; they are never silently dropped.
- Every agent handoff produces a traceable event record with trace_id and artifact references.
- At least one end-to-end feature flow can be executed through all specialized agents using the coordination layer as the gatekeeper.

13. Common Failure Modes

- Implicit state changes with no event record, making debugging impossible.
- Overly permissive routing that allows agents to work on stale or overlapping tasks.
- Coordination logic that becomes a hidden monolith of business rules and prompt hacks.
- Leases without expiration, causing dead tasks after agent failure.
- Parallelism without merge rules, producing conflicting artifacts that humans must manually untangle.
- Retry loops that reassign the same failing task without new evidence or changed strategy.

The biggest design trap is confusing “more agent calls” with “more progress.” Coordination must enforce evidence-based movement, not just activity.

14. Future Integration

- Part 13 Benchmark Harness will consume coordination events to measure throughput, retries, and handoff quality.
- Part 14 Deployment & Human Approval will depend on the coordination layer to block unsafe transitions until approval arrives.

- Part 15 Near-Codex Integrated System will rely on coordination to compose all subsystems into a single autonomous loop.
- Failure Recovery will use coordination events to choose the next strategy after a failed stage.
- Context Compression and Repository Intelligence will feed smaller, more relevant context bundles into the router and agents.

Coordination is the bridge that lets future parts reason over the same execution story instead of inventing their own private truths.

15. Completion Checklist

- Shared coordination contracts are implemented and versioned.
- Policy engine validates legal transitions and rejects invalid ones.
- Task registry tracks ownership, leases, dependencies, and artifacts.
- Router can dispatch single and parallel work items safely.
- Arbitration module can merge or escalate conflicting outputs deterministically.
- Event log records every important coordination action and can be replayed.
- Integration tests demonstrate end-to-end movement across multiple agents.
- Restart/replay tests reproduce the same state after process recovery.
- Telemetry shows traceable, debuggable agent handoffs.
- No undocumented coordination state remains outside the event/snapshot model.